RESEARCHLY AI · R26-IT-116

Module 2
Collaboration & Recommendation

Supervisor matching, peer group formation, aspect-based feedback sentiment, and supervisor effectiveness scoring — architecture, ML methodology, and calculation logic

Module owner: S. P. U. Gunathilaka

Service: `services/module2-collaboration` (FastAPI, port 8002)

Frontend: `apps/web/src/app/(dashboard)/collaboration/*`

Document generated: 27 August 2026

Contents

1 · Overview

Module 2 is the **collaboration layer** of the platform: it connects students to the right supervisor, to each other, and closes the feedback loop that lets the system learn how effective each supervisor actually is. It covers four user-facing flows:

1. **Supervisor matching** — rank SLIIT supervisors against a student's free-text research proposal.
2. **Supervisor profile view** — real publication history, charts, and an AI-written "recent focus" summary for a supervisor, pulled live from Semantic Scholar.
3. **Peer Connect** — students post open research-group slots and receive join requests from peers.
4. **Feedback & effectiveness** — students rate/comment on supervisors (with email OTP verification); aspect-based sentiment is extracted and rolled up into a per-supervisor effectiveness score.

All four live in one FastAPI microservice (`services/module2-collaboration` , port `8002`), consumed by four pages under `apps/web/src/app/(dashboard)/collaboration/` (`supervisor-match` , `peer-connect` , `feedback` , `effectiveness`).

> **Two different ML approaches coexist deliberately in this module:** supervisor matching uses a locally fine-tuned **SBERT bi-encoder** (fast, deterministic, explainable — the score a student sees is a real cosine similarity), while feedback sentiment and the "recent research focus" summary use **Gemini 2.5 Flash** (better suited to open-ended natural-language judgement calls that don't have a fixed answer). This mirrors Module 1's pattern of "retrieval where explainability matters, generative LLM where interpretation is genuinely needed."

2 · System Architecture & Data Flow

```
Next.js page (collaboration/*) │ fetch() with Supabase JWT ▼ Express API Gateway (apps/api-gateway, port 3001) │ requireAuth → mlRateLimiter → callMlService(2, path, method, body) ▼ FastAPI Module 2 service (services/module2-collaboration, port 8002) │ router → service layer (supervisor_matcher / supervisor_directory) ▼ ┌── models/trained_supervisor_matcher/ (fine-tuned SBERT, local) ├── data/supervisors_with_embeddings.json (pre-computed supervisor vectors) ├── Supabase Postgres (peer_groups, supervisor_ratings, profiles, supervisor_matches) ├── Semantic Scholar API (live publication data, rate-limited to 1 req/s) └── Gemini 2.5 Flash (sentiment analysis, research-focus summaries)
```

The gateway route file is `apps/api-gateway/src/routes/module2.routes.ts` (17 routes). All requests pass through `requireAuth` + `mlRateLimiter` and are proxied verbatim via `callMlService(2, path, method, body)` , with a 60-second timeout.

3 · Feature A — Supervisor Matching

Fine-tuned SBERT bi-encoder

What it does

A student writes a free-text research proposal (or, in the legacy path, a list of interest keywords + an abstract); `POST /matching/supervisors` returns the top-K SLIIT supervisors ranked by relevance, each with a similarity score, a blended "multi-factor" score, a human-readable explanation of why they matched, and their current availability/capacity.

Why a fine-tuned SBERT bi-encoder

Matching a proposal to a supervisor is fundamentally a semantic-similarity problem — "IoT-based crop monitoring using edge computing" should match a supervisor whose stated interests are "wireless sensor networks" even with zero keyword overlap. A bi-encoder (as opposed to a cross-encoder or an LLM call) lets every supervisor's embedding be pre-computed once and reused for every subsequent query — matching is then just a cosine similarity over pre-computed vectors, so it stays fast (no per-request model forward pass over supervisor text) and cheap enough to run on CPU for every student query. The base model is fine-tuned specifically on SLIIT supervisor↔proposal pairs (`training/train_supervisor_matcher.py` , `data/training_pairs.json`) so that domain-specific phrasing scores correctly, rather than relying on a generic sentence-similarity model.

How it is implemented

`app/services/supervisor_matcher.py` :

1. **Model loading** (`load_model`) — checks for `models/trained_supervisor_matcher/model.safetensors` ; if present, loads the fine-tuned SBERT (384-dim, `all-MiniLM-L6-v2` architecture); otherwise falls back to the base pretrained checkpoint. The router (`app/routers/supervisor.py`) accepts either a free-text `proposal` string (preferred) or the legacy `research_interests[] + abstract` shape, which is joined into an equivalent proposal string before encoding.
2. **Encoding & scoring** (`match_supervisors`) — the query is encoded once; supervisor embeddings are read from the pre-computed `data/supervisors_with_embeddings.json` (no query-time re-encoding of every supervisor's profile). Cosine similarity is computed manually: `dot(a,b) / (‖a‖·‖b‖ + 1e-8)` .
3. Candidates below `min_similarity` (default 0.2) are dropped.
4. **Multi-factor score** blends similarity with real-world availability so an excellent semantic match who is already at capacity doesn't crowd out a slightly-lower-similarity supervisor with open slots:

```
availability_factor = 0.0 if supervisor.availability == False 0.3 if current_students ≥
max_students (at capacity) 1.0 otherwise multi_factor_score = 0.70 · similarity + 0.30 ·
availability_factor
```

5. **Explanation generation** (`_build_explanation`) — a template sentence is chosen based on (a) whether any of the supervisor's stated interest keywords literally appear as a substring match in the query, and (b) similarity thresholds (>0.70 "excellent match", >0.55 "could supervise", else "some relevant expertise") — giving the student a reason, not just a number.

6. Results are sorted by `multi_factor_score` and truncated to `top_k` (default 5).

4 · Feature B — Supervisor Profile Enrichment

Semantic Scholar API | Gemini summary

What it does

Clicking a supervisor card opens a full profile: `GET /matching/supervisors/{id}/papers` returns their real publication list (title, year, venue, DOI/PDF link), a year-distribution bar chart, a research-topic pie chart (from their declared interests), and an AI-generated 2–3 sentence "recent research focus" summary with an activity-level tag.

Why an external API instead of a static dataset

Publication history changes continuously and a supervisor's actual output is the single strongest signal of what they can supervise *right now* — a static, hand-maintained list would go stale immediately. The Semantic Scholar Graph API is queried live by author name, giving up-to-date results without maintaining a separate database of publications.

How it is implemented (`app/routers/supervisor.py`)

- **Rate-limit-aware fetching** — Semantic Scholar's free/keyed tier allows only 1 request/second, shared across every concurrent caller of the process. A module-level `asyncio.Lock` plus a last-call timestamp (`_S2_LOCK` , `_S2_MIN_INTERVAL = 1.05s`) serialises every outbound call; `_s2_get` retries up to 3 times with increasing backoff on HTTP 429 before giving up. A single lookup needs two sequential calls — author search, then that author's papers.
- **Two-tier caching** (`_ss_cache`) — successful lookups (including genuinely-empty results) are cached for `_SS_TTL = 3600s` (1 hour); failed/rate-limited lookups are cached for only `_SS_FAILURE_TTL = 90s` so a transient 429 doesn't lock out a supervisor's data for an hour.
- **Year distribution & topic chart** — papers are bucketed by year (≥2010 only, to avoid a long tail skewing the chart) for the Recharts `BarChart` ; the supervisor's declared `research_interests` (top 10) feed the `PieChart` directly.
- **AI research-focus summary** (`_generate_research_focus`) — the 15 most recent paper titles + years are sent to Gemini with a prompt asking it to infer what the supervisor has recently worked on and how active they are, returned as JSON `{"summary", "recent_focus_areas": […], "activity_level": "Actively publishing" | "Occasional publications" | "Limited recent activity"}` . This is strictly best-effort — any Gemini failure just omits the field, it never breaks the papers/charts response — and is cached separately for 24 hours (`_FOCUS_TTL`) since a supervisor's overall research direction doesn't shift hour to hour.

5 · Feature C — Peer Connect (Group Formation)

Supabase CRUD | legacy SBERT matcher retained

What it does

A student can create a research group with a project title/description and a number of open slots (`POST /peers/groups`), browse open groups (`GET /peers/groups`), and request to join one (`POST /peers/groups/{id}/join-request`).

Why this shifted away from pure ML matching

The module still exposes a legacy content-based SBERT peer matcher (`POST /peers` , `app/routers/peer.py`) that embeds a student's interests/department and queries the `match_peers` pgvector RPC for similar student profiles. In production the primary flow is a more direct, human-in-the-loop **group marketplace** instead: students self-describe what they're building and who they need, rather than the system guessing compatibility purely from profile similarity — group formation depends on availability, timing and personal chemistry as much as topical overlap, which a similarity score alone can't capture well.

How the mailto join-request flow works

When a student requests to join a group, the backend persists the request to `peer_group_join_requests` and builds a `mailto:` URL with a pre-filled subject and body addressed to the group leader's contact email — the frontend opens this in a new tab so the email is actually sent from the *requester's own mailbox*, avoiding the need for a server-side SMTP relay for this flow.

Legacy SBERT peer matcher — how it works

```
query_text = ". ".join(student.research_interests) or student.department (fallback)
query_vec = embed(query_text) (services/shared/embedding_utils.py) candidates =
supabase.rpc("match_peers", {query_embedding, match_count: top_k×2, match_threshold: 0.3})
shared_interests = student_interests ∩ candidate_interests complementary_skills =
candidate_interests − student_interests
```

Results are sorted by similarity and truncated to `top_k` (default 10); each match reports both overlapping interests and complementary ones, since a good collaborator is often someone with adjacent, not identical, skills.

6 · Feature D — Feedback & Aspect-Based Sentiment

Gemini structured sentiment | OTP email verification

What it does

A student picks a supervisor (SLIIT-directory or system-registered), optionally verifies their email via a one-time code, then submits a 1–5 star rating with optional free-text feedback. If text is given, it is analysed for sentiment across four aspects — *methodology, writing, originality, data_analysis* — each labelled positive/neutral/negative with probability scores, plus an overall sentiment and a −1..+1 score.

Why Gemini for aspect-based sentiment

Aspect-based sentiment classification (deciding sentiment *per named dimension* within one free-text comment) needs genuine language understanding of which sentence relates to which aspect — a capability a general-purpose LLM already has well-calibrated, without needing a dedicated fine-tuned classifier trained on a small labelled dataset. The prompt (in `_gemini_sentiment`) explicitly asks for four aspect objects each with `{sentiment, positive, neutral, negative}` probabilities plus an overall verdict, returned as strict JSON via `shared/gemini_client.generate_json`.

How it is implemented (`app/routers/feedback.py`)

- **Fallback when Gemini/text unavailable** — `_stars_to_sentiment` derives a sentiment purely from the star rating: `score = (stars - 3) / 2`, giving −1 for 1★, 0 for 3★, +1 for 5★ — so every submitted rating always has a usable sentiment score even with empty feedback text.
- **Overall score/label reconciliation** — if Gemini's own `overall_sentiment` field is missing or invalid, it's derived from the mean of the four aspect scores: >0.3 → positive, <−0.3 → negative, else neutral.
- **OTP email verification** (`request-otp` / `verify-otp`) — a random 6-digit code is generated and kept in an in-memory dict (`_otp_store`, 10-minute TTL) keyed by lower-cased email. If SMTP env vars (`SMTP_HOST/USER/PASS`) are configured, the code is emailed via `smtplib` (SSL on port 465 or STARTTLS on 587, with a re-`ehlo()` after the TLS handshake as Gmail requires); otherwise the OTP is returned directly in the API response (`dev_otp`) so the demo still works without a configured mail relay. Verification is single-use — a successful check deletes the stored code.
- **Persistence** — `POST /feedback/submit` writes to `supervisor_ratings`, storing both the raw star rating and the computed sentiment (label + score + per-aspect probabilities) plus the verified rater name/email, so effectiveness scoring (Feature E) never has to re-run sentiment analysis.

7 · Feature E — Supervisor Effectiveness Scoring

Weighted aggregation, no ML at query time

What it does

The Effectiveness page lists every supervisor with a single blended 0–1 `overall_score`, sortable, plus a detail view (`GET /effectiveness/by-key`) showing the score breakdown, completion rate, average sentiment, student satisfaction rate, and the 10 most recent (verified) feedback entries.

Why a hand-weighted blend instead of a trained model

"Effectiveness" here is deliberately built as a transparent, auditable aggregation of signals already computed elsewhere (star ratings, Gemini sentiment scores from Feature D, and completion rate from Supabase) rather than a learned model — the weights are visible to the user in the API response itself (`breakdown.weights`), which matters for something that could affect a supervisor's reputation.

How it is implemented (`app/routers/effectiveness.py`)

```
star_norm = avg_stars / 5 (None if no ratings) sent_norm = (avg_sentiment + 1) / 2 (maps −1..+1 → 0..1) satisfaction = (# ratings with sentiment_score > 0.3) / (# ratings with a sentiment score) completion = completed_matches / total_matches (system supervisors only; 0 for SLIIT-directory entries) overall_score = Σ(weight × value) / Σ(weight) over whichever signals are available weights: stars 0.40 · sentiment 0.25 · satisfaction 0.15 · completion 0.20
```

Missing signals are simply dropped from both the numerator and denominator (re-normalised), so a supervisor with zero ratings still gets a defined score from completion rate alone rather than a hard zero. `_completion_rate_for_system` counts `supervisor_matches` rows with `status = "completed"` against the total for that supervisor — this only applies to system-registered supervisors, since SLIIT-directory entries have no match-tracking rows.

8 · Planned vs. Production Models

Spec model	File	Status	What runs in production instead
Aspect-based sentiment — fine-tuned `bert-base-uncased`, 4 aspects (methodology/writing/originality/data_analysis), target accuracy ≥93%	`app/models/sentiment.py`	Wrapper class present; no trained weights bundled; not imported by any router	Gemini 2.5 Flash structured-JSON prompt (Feature D)

> **Why this matters for the report:** the fine-tuned BERT classifier would need a labelled SLIIT-specific dataset large enough to hit the 93% accuracy target across four aspects simultaneously; Gemini's zero-shot aspect classification reaches comparable practical quality without that labelling effort, and — as with Module 1 — it lets the same underlying LLM client be reused across the mind-map, proposal-fallback, and sentiment features rather than maintaining a separate model artifact.

9 · Endpoint Reference

Method & Path (via gateway `/api/v1/module2/…`)	Feature	Backing service
POST `/matching/supervisors`	Supervisor matching	supervisor_matcher.py
GET `/matching/supervisors/{id}/papers`	Profile + publications + charts	Semantic Scholar + Gemini
POST `/peers/groups`	Create group	Supabase (peer_groups)
GET `/peers/groups`	List groups	Supabase (peer_groups)
GET `/peers/groups/{id}`	Group detail	Supabase (peer_groups)
POST `/peers/groups/{id}/join-request`	Join request + mailto	Supabase (peer_group_join_requests)
POST `/peers` *(legacy)*	Content-based peer matching	embedding_utils.py + pgvector RPC
POST `/feedback/request-otp`	Email OTP request	smtplib / in-memory store
POST `/feedback/verify-otp`	Email OTP verify	in-memory store
POST `/feedback/analyze`	Standalone sentiment analysis	Gemini 2.5 Flash
GET `/feedback/supervisors`	Rateable supervisor directory	supervisor_directory.py
POST `/feedback/submit`	Submit rating + feedback	Gemini + Supabase (supervisor_ratings)
GET `/feedback/by-supervisor`	Ratings for one supervisor	Supabase (supervisor_ratings)
GET `/effectiveness`	Effectiveness list (all supervisors)	effectiveness.py
GET `/effectiveness/by-key`	Effectiveness detail	effectiveness.py
GET `/effectiveness/{id}` *(legacy)*	Effectiveness by system UUID	effectiveness.py